Лабораторная работа № 1

Reliable Remote Calls

Адаптивная политика тайм-аутов и повторных попыток

Пошаговое описание работы решения и выполненной работы

Решение	RRC.Student/AdaptiveRequestPolicy.cs
Платформа	.NET 8, C#
Стенд проверки	RRC.Bench (12 сценариев, 20 seed)
Результат на открытом сценарии	100 % успешных, среднее ≈ 100 мс, p95 ≈ 145 мс, 1.000 попытки/запрос

1. Цель работы

В репозитории находится эмулятор удалённого сервера. Запросы приходят с заданной интенсивностью, сервер отвечает через случайное время, а при большом числе одновременных попыток работает медленнее. Нужно реализовать класс, реализующий интерфейс **IRequestPolicy**, который:

- выбирает тайм-аут для каждой попытки (метод **GetTimeout**);
- по результату попытки решает, нужна ли повторная попытка и через сколько (метод **OnResult**).

Цель — выполнить успешно как можно больше запросов, сохраняя небольшое время ответа и малое число попыток. Оценка ведётся по четырём метрикам (по убыванию веса): доля успешных запросов, среднее время и p95, число попыток на запрос. Оценивание проводится на шести **закрытых** сценариях с другими параметрами, поэтому решение не должно быть подогнано под числа открытого примера.

Важная особенность: **тайм-аут не останавливает обработку на сервере**. Повторная попытка — это дополнительная нагрузка, которая может замедлить все остальные запросы.

2. Структура решения

Проект	Назначение
RRC.Abstractions	Интерфейсы и модели данных: IRequestPolicy, RequestContext, RequestObservation, RetryDecision, модели поступления, задержки и нагрузки.
RRC.Simulation	Движок дискретно-событийной симуляции (SimulationRunner) и расчёт метрик.
RRC.Simulation.Public	Открытый сценарий и простые модели: постоянный поток, равномерная задержка, модель ёмкости.
RRC.Student	Место для решения. Добавлен класс **AdaptiveRequestPolicy**.
RRC.Runner	Консольный запуск симуляции. В Program.cs выбрана AdaptiveRequestPolicy.
RRC.Bench (новый)	Стенд для проверки политики на 12 разных сценариях и нескольких seed. В .sln не добавлен, запускается отдельно.

Проекты RRC.Evaluation и RRC.Simulation.Private упомянуты в RRC.sln, но в репозитории отсутствуют (используются только преподавателем). Поэтому сборка всего .sln из командной строки завершается ошибкой, а собирать нужно конкретный проект (см. раздел 8).

3. Как работает эмулятор (пошагово)

Симуляция дискретно-событийная: события лежат в очереди с приоритетом по времени, время «перепрыгивает» от события к событию. Все случайные величины берутся из трёх независимых генераторов, созданных из seed, поэтому прогон воспроизводим.

1. **NewLogicalRequest.** Создаётся логический запрос с номером RequestId и временем создания. Сразу планируется его первая попытка, а следующий запрос планируется через интервал от модели поступления (в открытом сценарии — каждые 10 мс, т.е. 100 запросов/с в течение 30 с).
2. **StartAttempt.** Вызывается **policy.GetTimeout(requestId, attempt)**. Модель задержки выдаёт базовое время (открытый сценарий: равномерно 50..150 мс), модель нагрузки умножает его на **max(1, in-flight / capacity)**. Полученная задержка **фиксируется в момент входа**. Планируются два события: завершение на сервере и тайм-аут.
3. **ServerProcessingComplete.** Счётчик in-flight уменьшается. Если тайм-аут этой попытки ещё не наступил — запрос успешен, его логическая задержка (от создания запроса, включая все предыдущие попытки и паузы) записывается в статистику, политике передаётся Success. Если тайм-аут уже был — это **LateResponse**: работа сервера потрачена впустую, политике ничего не сообщается.
4. **AttemptTimeout.** Если ответ ещё не пришёл, попытка считается неудачной, но **сервер продолжает её обрабатывать**. Политике передаётся Timeout; если она вернула RetryAfter(delay), через delay запускается новая попытка, иначе запрос считается проваленным.
5. **Окончание.** Через 40 с симуляция останавливается; все незавершённые запросы считаются проваленными. Считаются метрики: доля успехов, среднее и p95 задержки успешных запросов, попыток на запрос.

Ключевые наблюдения из анализа кода

- Политика видит только **Elapsed** попытки: для успеха — точное время, для тайм-аута — само значение тайм-аута. Опоздавшие ответы не сообщаются, т.е. данные **цензурированы справа**: про тайм-аут известно лишь «задержка ≥ T».
- У политики нет часов симуляции, но есть номера запросов (идут по порядку поступления).
- Тайм-аут не отменяет работу: каждая повторная попытка увеличивает in-flight. При in-flight > capacity все новые попытки замедляются пропорционально, а значит появляются новые тайм-ауты и новые ретраи — **положительная обратная связь (retry storm)**.
- В открытом сценарии in-flight ≈ 100 зап/с × 0.1 с = 10 < 20, т.е. нагрузка не влияет. Пример SampleRequestPolicy с фиксированным тайм-аутом 120 мс обрывает ≈ 30 % попыток (задержки до 150 мс), делает только один ретрай и теряет ≈ 8–9 % запросов.
- Отказ от ретрая всегда означает провал запроса, а провал — самая «дорогая» метрика. Поэтому выгоднее делать тайм-ауты редкими, а не отказываться от повторов.

4. Решение: AdaptiveRequestPolicy

Идея: тайм-аут выбирается не «на глаз», а по наблюдаемому распределению задержек. Повторы используются только когда статистика доказывает, что перезапуск медленной попытки быстрее, чем ожидание, и что медленность случайна, а не вызвана нагрузкой. Во всех остальных случаях тайм-аут держится выше всех наблюдаемых задержек и лишь страхует от редких выбросов.

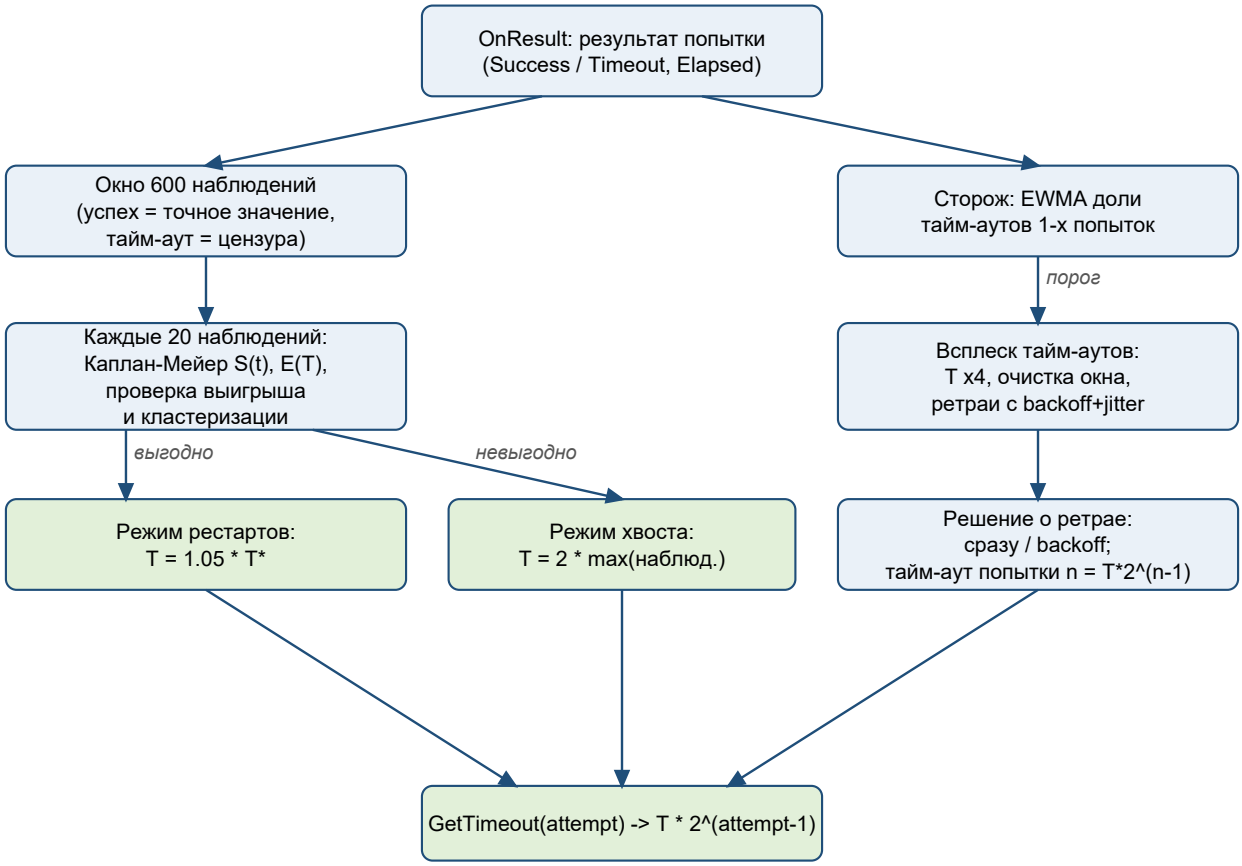


Рис. 1. Схема работы политики

Шаг 1. Сбор наблюдений

Каждый вызов OnResult добавляет в кольцевое окно из 600 последних наблюдений запись (RequestId, номер попытки, значение, признак цензуры). Успех — точное значение задержки, тайм-аут — цензурированное значение «не меньше T».

Шаг 2. Оценка распределения методом Каплана–Мейера

Раз в 20 наблюдений окно сортируется, и строится оценка функции выживания S(t) = P(задержка > t). Метод Каплана–Мейера корректно учитывает цензурированные значения: тайм-аут не считается ни успехом, ни «бесконечной» задержкой, а просто выводит запрос из группы риска.

```
S(t) = Π  (1 - 1 / n_i)   по всем успехам с задержкой t_i <= t,
       n_i — число наблюдений, ещё «в риске» к моменту t_i
```

Шаг 3. Выбор оптимального тайм-аута

Если попытки независимы и после тайм-аута T сразу запускается новая, ожидаемое время до успеха (классическая формула стратегии перезапусков):

```
E(T) = [ интеграл от 0 до T функции S(t) dt ] / (1 - S(T))
среднее число попыток = 1 / (1 - S(T))
```

- E(T) считается для каждого наблюдённого значения T, при котором доля тайм-аутов S(T) ≤ 15 %.
- Из почти оптимальных T (E ≤ 1.05·min E) берётся **наибольший** — та же задержка, но меньше ретраев.
- Перезапуски включаются, только если они дают выигрыш ≥ 10 % относительно нижней оценки среднего без перезапусков (площадь под S(t)). Тогда тайм-аут = 1.05·T* (**режим рестартов**).
- Иначе (**режим хвоста**) тайм-аут = 2 × максимальная наблюдённая задержка: тайм-ауты почти не происходят, но хвост распределения остаётся видимым для статистики.

Шаг 4. Проверка независимости (защита от нагрузки)

Формула E(T) верна, только если медленность случайна. Если медленность вызвана нагрузкой или временной деградацией, повтор будет таким же медленным и лишь добавит нагрузку. Политика проверяет это двумя способами:

- **Кластеризация:** среди первых попыток соседних по RequestId запросов сравнивается P(медленный | предыдущий медленный) с общей долей медленных. Для независимых задержек они равны. Если условная вероятность больше 2·p + 0.1 (и таких пар не меньше трёх) — перезапуски запрещаются. Сравниваются только действительно соседние номера: медленные ответы приходят поздно, и в окне оказываются старые запросы, чьи соседи уже ушли из окна.
- **Проверка реальных повторов:** отслеживается доля вторых попыток, завершившихся быстрее T. Если после 20 повторов она меньше половины предсказанной (1 − S(T)), доверие к перезапускам отключается до конца прогона.

Шаг 5. Сторож перегрузки

Экспоненциальное скользящее среднее (α = 0.05) доли тайм-аутов первых попыток. Если оно превысило max(0.25, 2·ожидаемая доля + 0.15), значит задержки резко выросли: тайм-аут сразу умножается на 4, окно очищается (старые данные описывают другой режим и тянули бы тайм-аут вниз), а повторы запускаются с экспоненциальной задержкой и случайным разбросом (full jitter) — чтобы не создать шторм повторов. Между срабатываниями — минимум 10 наблюдений.

Шаг 6. Асимметричное изменение тайм-аута

Тайм-аут растёт мгновенно, а уменьшается не более чем на 15 % за пересчёт. Причина — систематическая ошибка выборки: быстрые ответы всегда приходят первыми, а медленные ещё в пути, поэтому свежая статистика оптимистична. Начальный тайм-аут — 10 с (пока данных нет, безопаснее ждать).

Шаг 7. Повторы

- Запрос не бросается досрочно: до 8 попыток, тайм-аут попытки n равен $T \cdot 2^{n-1}$ (не более 60 с), поэтому даже при неверной оценке запрос в итоге дождётся ответа.
- В нормальном режиме повтор запускается сразу (ожидание только увеличивает задержку при случайном выбросе); при перегрузке — через случайную паузу из [0, тайм-аут попытки].
- Переданный объект Random используется только для jitter, поэтому результаты воспроизводимы по seed.

Параметры

Параметр	Значение	Смысл
WindowSize / RecomputeEvery	600 / 20	размер окна и частота пересчёта
MinSamples	60	минимум данных для оценки
InitialTimeoutMs	10 000	тайм-аут до появления статистики
MaxRestartShare	0.15	макс. доля тайм-аутов в режиме рестартов
RequiredGain / NearOptimum	0.9 / 1.05	мин. выигрыш 10 %; допуск к оптимуму 5 %
TailMargin / InteriorMargin	2 / 1.05	запас тайм-аута в двух режимах
MaxDecrease	0.85	макс. уменьшение тайм-аута за пересчёт
MaxClustering / MinClusterEvidence	2 / 3	порог кластеризации медленных запросов
GuardAlpha / GuardMinRate / GuardGrowth	0.05 / 0.25 / ×4	параметры сторожа
MaxAttempts	8	предел попыток (тайм-ауты удваиваются)

5. Что было сделано (ход работы)

1. **Анализ.** Изучены README, интерфейсы RRC.Abstractions, движок SimulationRunner и открытый сценарий. Выявлены ключевые свойства: цензурированные наблюдения, неотменяемая работа сервера, фиксация задержки при входе, обратная связь через in-flight.
2. **Базовые измерения.** SampleRequestPolicy на открытом сценарии: ≈ 91 % успехов, среднее 134 мс, p95 ≈ 323 мс, 1.29 попытки на запрос.
3. **Стенд RRC.Bench.** Так как закрытые сценарии неизвестны, создан стенд с 12 сценариями: тяжёлые и бимодальные хвосты, редкие огромные выбросы, логнормальное распределение, волнообразная задержка, пуассоновский поток со всплесками, временная деградация, работа у предела и сверх ёмкости, очень быстрый и очень медленный сервис. Для сравнения добавлены политики: Sample, «фиксированный тайм-аут 200 мс + 3 повтора», «без повторов с тайм-аутом 10 с / 60 с».
4. **Версия 1** (Каплан–Мейер + E(T) + сторож). Отлично работала на хвостах, но проигрывала стратегии «без повторов» на всплесках и деградации: повторы во время перегрузки усугубляли её.
5. **Исправление 1.** Порог выигрыша 10 %, ограничение доли тайм-аутов 15 %, проверка независимости реальных повторов, тест кластеризации медленных запросов.
6. **Исправление 2.** Обнаружена систематическая ошибка «быстрые ответы приходят первыми»: на старте тайм-аут падал слишком низко перед всплеском. Введено медленное уменьшение тайм-аута и начальное значение 10 с.
7. **Исправление 3.** Тест кластеризации давал ложные срабатывания: поздно пришедшие медленные ответы старых запросов оказывались соседями после сортировки. Теперь сравниваются только запросы с номерами, отличающимися ровно на 1, и требуется минимум 3 события.
8. **Подбор параметров сторожа** (α, пауза, множитель) — оставлен вариант с наибольшей долей успехов.
9. **Интеграция.** В RRC.Runner/Program.cs выбрана AdaptiveRequestPolicy; финальная проверка на 20 seed.

6. Результаты

Открытый сценарий (RRC.Runner)

Seed	Политика	Успех	Среднее, мс	p95, мс	Попыток/запрос
42	Sample	91.8 %	134.22	322.57	1.286
42	Adaptive	100.0 %	99.41	144.90	1.000
1	Sample	91.3 %	134.81	324.31	1.296
1	Adaptive	100.0 %	99.57	144.87	1.000
2	Sample	91.2 %	134.32	325.46	1.292
2	Adaptive	100.0 %	99.95	144.54	1.000

Все запросы успешны, задержка равна чистой задержке сервера (минимально возможной), лишних попыток нет. Политика обнаружила, что распределение ограничено (≤ 150 мс) и перезапуски бесполезны, поэтому работает в режиме хвоста.

Стенд RRC.Bench: средние значения по 20 seed

Сценарий	Политика	Успех	Среднее, мс	p95, мс	Попыток
Публичный сценарий (100 зап/с, 50..150 мс, ёмкость 20)	Sample	91.2 %	136.4	325.0	1.301
	200 мс + 3 повтора	100.0 %	100.0	145.0	1.000
	Без повторов, 60 с	100.0 %	100.0	145.0	1.000
	Adaptive	100.0 %	100.0	145.0	1.000
Бимодальный хвост: 5% запросов 1..3 с, ёмкость 20	Sample	5.2 %	142.3	327.6	1.960
	200 мс + 3 повтора	32.2 %	138.8	369.2	3.060
	Без повторов, 60 с	100.0 %	261.2	787.5	1.000
	Adaptive	100.0 %	239.3	452.1	1.047
Бимодальный хвост, ёмкость 40	Sample	88.8 %	140.3	325.8	1.335
	200 мс + 3 повтора	100.0 %	113.2	234.8	1.052
	Без повторов, 60 с	100.0 %	194.0	539.7	1.000
	Adaptive	100.0 %	126.0	183.3	1.045
Редкие выбросы: 1% запросов 5..8 с	Sample	100.0 %	42.2	58.4	1.010
	200 мс + 3 повтора	100.0 %	42.5	58.4	1.010
	Без повторов, 60 с	100.0 %	105.5	58.4	1.000
	Adaptive	100.0 %	45.5	58.4	1.010
Волна задержки: x(1 ± 0.8) с периодом 10 с	Sample	34.9 %	82.6	307.4	1.692
	200 мс + 3 повтора	5.9 %	159.1	399.3	3.829
	Без повторов, 60 с	100.0 %	100.1	224.4	1.000
	Adaptive	100.0 %	100.1	224.4	1.000
Логнормальная задержка (медиана 80 мс, σ = 1)	Sample	88.5 %	112.3	306.4	1.345
	200 мс + 3 повтора	99.9 %	130.7	391.9	1.218
	Без повторов, 60 с	100.0 %	132.3	418.8	1.000
	Adaptive	100.0 %	130.3	414.0	1.012
Пуассон + всплески 400 зап/с на 1 с каждые 5 с	Sample	0.5 %	89.0	115.2	1.995
	200 мс + 3 повтора	1.3 %	132.1	192.7	3.961
	Без повторов, 60 с	100.0 %	445.3	1133.2	1.000
	Adaptive	100.0 %	445.3	1133.2	1.000
Деградация: с 10 по 15 с задержка x8	Sample	30.3 %	135.9	324.5	1.768
	200 мс + 3 повтора	33.3 %	100.0	145.1	3.000
	Без повторов, 60 с	100.0 %	2276.8	10191.5	1.000
	Adaptive	99.8 %	2653.4	11782.1	1.014
Почти предельная нагрузка: 180 зап/с	Sample	1.3 %	122.0	321.6	1.989
	200 мс + 3 повтора	100.0 %	101.2	146.6	1.000
	Без повторов, 60 с	100.0 %	101.2	146.6	1.000

Сценарий	Политика	Успех	Среднее, мс	p95, мс	Попыток
	Adaptive	100.0 %	101.2	146.6	1.000
Медленный сервис: 500..1500 мс, 20 зап/с	Sample	0.0 %	—	—	2.000
	200 мс + 3 повтора	0.0 %	—	—	4.000
	Без повторов, 60 с	100.0 %	1297.3	2050.9	1.000
	Adaptive	100.0 %	1297.3	2050.9	1.000
Быстрый сервис: 2..10 мс, 1000 зап/с, ёмкость 10	Sample	100.0 %	6.0	9.6	1.000
	200 мс + 3 повтора	100.0 %	6.0	9.6	1.000
	Без повторов, 60 с	100.0 %	6.0	9.6	1.000
	Adaptive	100.0 %	6.0	9.6	1.000
Перегрузка: 220 зап/с (больше ёмкости)	Sample	0.7 %	105.5	264.5	1.993
	200 мс + 3 повтора	2.8 %	144.1	315.5	3.919
	Без повторов, 60 с	100.0 %	1545.0	3375.7	1.000
	Adaptive	100.0 %	1545.0	3375.7	1.000

Выводы по стенду

- Adaptive ни в одном сценарии не опустилась ниже 99.8 % успехов (100 % везде, кроме синтетической деградации). Такую же надёжность даёт только «без повторов, 60 с», но она заметно медленнее на хвостах.
- Политики с фиксированным тайм-аутом катастрофически проигрывают при медленном сервисе, всплесках и перегрузке (вплоть до 0–3 % успехов) — именно из-за шторма повторов.
- На тяжёлых хвостах Adaptive заметно лучше стратегии «без повторов»: бимодальный хвост с ёмкостью 40 — среднее 126 мс против 194 мс, p95 183 мс против 540 мс; редкие выбросы — 45 мс против 105 мс; дополнительных попыток лишь ≈ 1–5 %.
- В сценариях, где медленность вызвана нагрузкой (всплески, перегрузка, волна), Adaptive сама переходит к поведению «ждать и не повторять» и совпадает с лучшей возможной стратегией.
- Единственный сценарий, где Adaptive уступает «без повторов, 60 с» — резкая деградация x8: несколько десятков тайм-аутов до срабатывания сторожа добавляют нагрузку (≈ 1.4 % лишних попыток).

7. Как запустить

Требуется .NET 8 SDK/Runtime (подойдёт и более новый SDK, собирающий net8.0).

Visual Studio

1. Открыть RRC.sln (отсутствующие проекты RRC.Evaluation и RRC.Simulation.Private будут помечены как недоступные — это нормально).
2. Сделать RRC.Runner запускаемым проектом. В RRC.Runner/Program.cs уже указано **new AdaptiveRequestPolicy()**.
3. Запустить, ввести seed и число запросов для трассировки (Enter — значения по умолчанию).

Командная строка

```
dotnet build RRC.Runner
dotnet run --project RRC.Runner -- --seed 42
dotnet run --project RRC.Runner -- --seed 42 --trace 10

# стенд: 12 сценариев x 5 политик, аргумент — число seed
dotnet run --project RRC.Bench -- 20
```

Для сдачи передаётся один файл: **RRC.Student/AdaptiveRequestPolicy.cs**. Он зависит только от RRC.Abstractions; все вспомогательные типы вложены в класс.

8. Ограничения и итог

- Политика предполагает, что задержки в пределах окна (≈ 600 наблюдений) приблизительно стационарны; резкие изменения обрабатывает сторож, плавные — скользящее окно.
- При внезапной деградации неизбежны несколько десятков тайм-аутов до срабатывания сторожа.
- Все константы выбраны из соображений устойчивости, а не подгонки под открытый сценарий: на нём результат определяется только тем, что тайм-аут выше максимальной задержки.

Итог. Цель лабораторной работы достигнута: реализована стратегия, которая на открытом сценарии даёт 100 % успешных запросов с минимально возможной задержкой и без лишних попыток (против ≈ 91 % у примера), а на 12 разнородных стресс-сценариях устойчиво сохраняет долю успехов, используя повторы только там, где они статистически выгодны и не создают шторм нагрузки.